Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Lecture 4

- Recommended Reading:
  - Chap. 4

- Solving the recurrence for the runtime of Mergesort using a recursion tree
- Solving recurrences using induction

Recurrence relation for runtime of Mergesort

- $T(n) = 2T(n/2) + c_2 n \qquad \text{if } n > 1$
- $T(n) = c_1 \qquad \text{if } n = 1$

Section 2.3 of textbook shows solution to recurrence for Mergesort

Recurrence relation for runtime of Mergesort

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$
  - For simplicity, set $c_1 = c_2 = 1$

Method 1: Using Recursion Tree to Solve Recurrence

Recursion Tree method

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$ (assuming n a power of 2)

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n	n
1	n/2	2	n/2	n
2	n/4	4	n/4	n
⋮				
i	$n/2^i$	2^i	$n/2^i$	n
⋮				
$\log_2 n - 1$	2	$n/2$	2	n
$\log_2 n$	1	n	1	n

per level

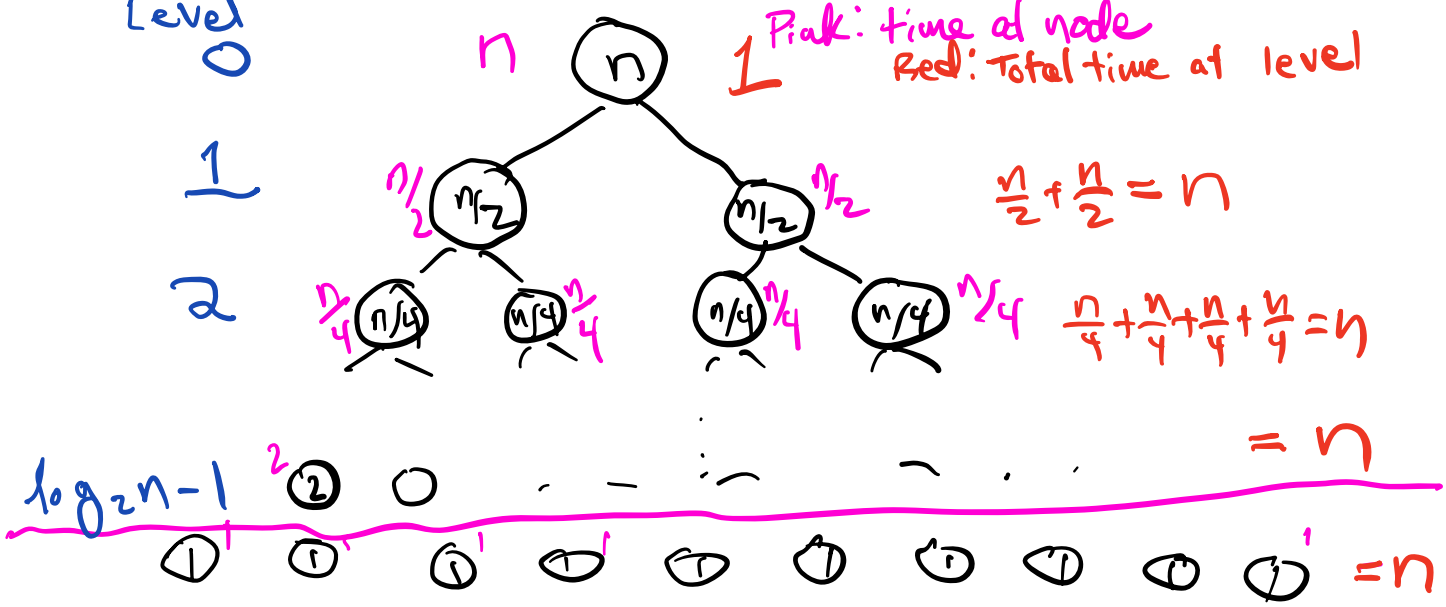
Total time all levels

$= (\log_2 n) * n + n$

Total time at base

#levels before base

Time per level before base


Level
0
1
2
n
n
1
Pink: time of node
Red: Total time at level
n/2
n/2
n/2
n/2
n/4
n/4
n/4
n/4
n/4
n/4
n/4
n/4
n/2 + n/2 = n
n/4 + n/4 + n/4 + n/4 = n
= n
log2n-1
2
1
1
1
1
1
1
1
1
1
1
=n

Recursion Tree method

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$

Level #	Input size	#nodes	Time per node	Time in level
0	n	1	n	
1	n/2	2	n/2	
2	n/4	4	n/4	
⋮				
i	$n/2^i$	2^i	$n/2^i$	
⋮				
	2	$n/2$	2	
	1			

T(n)=

Recursion Tree method

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$

Level #	Input size	#nodes	Time per node	Time in level
0	n	1	n	n
1	n/2	2	n/2	n
2	n/4	4	n/4	n
⋮				⋮
i	$n/2^i$	2^i	$n/2^i$	n
⋮				⋮
$\log_2 n - 1$	2	$n/2$	2	n
$\log_2 n$	1	n	1	n

T(n)= $n \log_2 n + n$ Check it?

Using recurrence

$T(1) = 1$

$T(2) = 2T(1) + 2 = 4$

$T(4) = 2T(2) + 4 = 2 * 4 + 4 = 12$

$T(8) = 2T(4) + 8 = 2 * 12 + 8 = 32$

Compare to formula

$T(n) = n \log_2 n + n$ ✓

$T(1) = 1 * 0 + 1 = 1$ ✓

$T(2) = 2 \log_2 2 + 2 = 4$

$T(4)$ ✓

$T(8) = 8 \log_2 8 + 8 = 32$ ✓

Recurrence relation for runtime of Mergesort

- $T(n) = c_1$ if $n = 1$
- $T(n) = 2T(n/2) + c_2 n$ if $n > 1$
  - ~~For simplicity, set $c_1 = c_2 = 1$~~
  - Re-do, with c_1 and c_2

Recursion Tree method

- $T(n) = c_1$ if $n = 1$
- $T(n) = 2T(n/2) + c_2 n$ if $n > 1$

$T(n) = \Theta(1)$ for $n = 1$

$T(n) = 2T(n/2) + \Theta(n)$ for $n > 1$

Level #	Input size	#nodes	Time per node	Time in level
0	n	1	$c_2 n$	$c_2 n$
1	n/2	2	$c_2 n/2$	$c_2 n$
2	n/4	4	$c_2 n/4$	$c_2 n$
⋮				
i	$n/2^i$	2^i	$c_2 n/2^i$	
⋮				
$\log_2 n - 1$	2	$n/2$	$c_2 2$	$c_2 n$
$\log_2 n$	1	n	c_1	$c_1 n$

$$T(n) = c_2 n \log_2 n + c_1 n \leq (c_1 + c_2) n \log_2 n$$

$\geq c_2 n \log_2 n$

Quick Review: Proof by Induction

Proof by (Weak) Induction

- Want to prove a proposition $P(n)$ is true for all integers $n \geq 1$
- Proposition $P(n)$ is a statement about n that is either True or False, e.g.,
  - $\sum_{i=1}^{n} i = n(n+1)/2$
  - $n^2 \geq 8n + 1$
- Approach: "Weak" induction
  - Base Case: Prove $P(n)$ is true for $n = 1$
  - Let n be an integer such that $n > 1$
  - Induction Hypothesis:
    - Assume $P(n-1)$ is true
  - Induction Step:
    - Show, using the induction hypothesis, that $P(n)$ is true

Principle of Weak Induction

- Base case shows $P(n)$ is true for $n = 1$
- Since it's true for $n = 1$, induction step tells us it's true for $n = 2$
- Since it's true for $n = 2$, induction step tells us it's true for $n = 3$
- …

~~Base case other than n=1~~

- Proof by "Weak" Induction
- Want to prove a proposition $P(n)$ is true for all integers $n \geq 10$
- Proposition $P(n)$ is a statement about n that is either True or False, e.g.,
  - $\sum_{i=1}^{n} i = n(n+1)/2$
  - $n^2 \geq 8n + 1$
- Approach: "Weak" induction
  - Base Case: Prove $P(n)$ holds for $n = 10$
  - Let n be an integer such that ~~$n \geq 1$~~ $n > 10$
  - Induction Hypothesis:
    - Assume $P(n-1)$ holds
  - Induction Step:
    - Show, using the induction hypothesis, that $P(n)$ holds

Example of proof by (Weak) Induction

- Prove that $n! < n^n$, for all integers $n \geq 2$ using proof by induction
  - What is $P(n)$?
- Base Case: Prove $P(n)$ holds for $n = 2$
  - When $n = 2$, $n! = 2$ and $n^n = 4$ $2 < 4$ ✓
- Let n be an integer such that $n > 2$
- Induction Hypothesis:
  - Assume $P(n-1)$ is true: $(n-1)! < (n-1)^{n-1}$
- Induction Step:
  - Show, using the induction hypothesis, that $P(n)$ holds
  - $n! = (n-1)! * n < (n-1)^{(n-1)} * n$ from inductive hyp
    $< n^{(n-1)} * n$
    $= n^n$

$n! = 1 \cdot 2 \cdot 3 \cdots n$

$n^n = n \cdot n \cdot n \cdots n$

- Didn't really need to do proof by induction to see that
  $n! \leq n^n$
- (Just an quick review of proof by induction)

- Sometimes need to use Strong Induction

Proof by Strong Induction

- Want to prove $P(n)$ is true for all integers $n \geq 1$
- Approach: Strong induction [use a stronger induction hypothesis]
  - Base Case: Prove $P(n)$ is true for $n = 1$
  - Let n be an integer such that $n > 1$
  - Induction Hypothesis:
    - Assume P(k) is true for k=1, 2, 3, 4, …, $n - 1$
  - Induction Step:
    - Show, using the induction hypothesis, that $P(n)$ is true

Method 2 for Solving Recurrences: Using Guessing + Pf by Induction

Book calls this the Substitution Method

Inductive approach

- Now use a different approach to (again) show that the running time of Mergesort is $O(n \log n)$
- Alternative to solving recurrence using recursion tree method

Inductive approach to solving recurrence relations

- Only trying to get asymptotic solution, e.g., $T(n) = O(n \log n)$, not $T(n) = n \log n + n$
- First guess solution
- Then try to prove correct using (strong) induction,
  e.g. Suppose recurrence is
  - $T(n) = 2T(\lfloor n/2 \rfloor) + n \qquad \text{if } n > 1$
    $T(1) = 1$
- Guess: $T(n) = O(n \log n)$ (we'll only show big-Oh)
- Show holds by (strong) induction

Actual mergesort recurrence

$$T(n) = T(\lfloor \tfrac{n}{2} \rfloor) + T(\lceil \tfrac{n}{2} \rceil) + \Theta(n)$$

- $T(n) = 2T(\lfloor n/2 \rfloor) + n \qquad \text{if } n > 1$
  $T(1) = 1$
- Want to show that $T(n) = O(n \log n)$
- That is, want to prove the following
  Claim: There exist $c > 0$, $n_0 > 0$ such that $T(n) \leq cn \log_2 n$ for all $n \geq n_0$

- Claim: There exist $c > 0$, $n_0 > 0$ such that $T(n) \leq cn \log_2 n$ for all $n \geq n_0$
- Use induction to prove the stronger claim (choosing $c = 5$ and $n_0 = 2$):
- Claim: $T(n) \leq 5n \log_2 n$ for all $n \geq 2$

Proof of Claim by induction

- $T(n) = 2T(\lfloor n/2 \rfloor) + n \quad \text{if } n > 1$
  $T(n) = 1 \quad \text{if } n = 1$
- **Claim:** $T(n) \leq 5n \log_2 n$ for all $n \geq 2$
- **Proof:** By induction on n
- **Base case:**
  - $T(2) = 2T(1) + 2 = 4$
  - $5 * 2 \log_2 2 = 10$
  - $4 < 10$



- $T(n) = 2T(\lfloor n/2 \rfloor) + n \quad$ if $n > 1$
  $T(n) = 1 \quad$ if $n = 1$
- **Claim:** $T(n) \leq 5n \log_2 n$ for all $n \geq 2$
- **Proof:** By induction on n
- **Base case:**
  - $T(2) = 2T(1) + 2 = 4$
  - $4 \leq 4 \log_2 4 = 8$
- Let $n > 2$
- **Inductive hypothesis:** Suppose $T(k) \leq 5k \log_2 k$ holds for $k = 2, 3, \ldots, n-1$
  - strong induction, we don't just assume true for $k = n - 1$

- **Inductive hypothesis:** Suppose $T(k) \leq 5k \log_2 k$ holds for $k = 2, 3, \ldots, n-1$
- **Induction step:** Using inductive hypothesis, show that $T(n) \leq 5n \log_2 n$
  - Equivalently, need to show $2T(\lfloor n/2 \rfloor) + n \leq 5n \log_2 n$ since $T(n) = 2T(\lfloor n/2 \rfloor) + n$
    - $2T(\lfloor n/2 \rfloor) + n \leq 2 * 5(\lfloor n/2 \rfloor)\log_2(\lfloor n/2 \rfloor) + n$ by Induction Hypothesis

$$\leq 10(n/2)\log_2(n/2) + n$$
$$= 5n \log_2 n - 5n + n$$
$$\leq 5n \log_2 n$$

$5n \log_2 \frac{n}{2} = 5n \log_2 n - 5n \log_2 2$

$= 5n \log_2 n - 5n$

- **So done?**
- Actually not, induction step doesn't work for $n = 3$
- We said:
  - $2T(\lfloor n/2 \rfloor) + n \leq 5 * 2(\lfloor n/2 \rfloor)\log_2(\lfloor n/2 \rfloor) + n$ by Induction Hypothesis
  - But when $n = 3$, $\lfloor n/2 \rfloor = 1$
  - Inductive hypothesis doesn't tell us anything about $T(1)$
    - It isn't true that $T(n) \leq 5n \log_2 n$ when $n = 1$

How do you fix this?

- Have additional base case for $n = 3$. Recall
  - $T(n) = 2T(\lfloor n/2 \rfloor) + n \qquad \text{if } n > 1$
  - Proposition is that $T(n) \leq 5n \log_2 n$ for $n \geq 2$.
- Base Cases
  - $T(2) = 2T(1) + 2 = 4$
    - $4 \leq 5 * 2 \log_2 2$
  - $T(3) = 2T(1) + 3 = 5$
    - $5 \leq 5 * 3 \log_2 3$
- Use inductive hypothesis for $n > 3$,
  Assume $T(k) \leq 5n \log_2 n$ for $k = 2, 3, 4, \ldots, n-1$
- Don't need any additional base cases: For $n > 3$, $\lfloor n/2 \rfloor \geq 2$ so can apply induction hypothesis to $T(\lfloor n/2 \rfloor)$

Base $n = 3$

Let $n = 4$

$T(\lfloor \frac{n}{2} \rfloor)$

- We now have proof that $T(n) \leq 5n \log_2 n$ for all $n \geq 2$
  - $T(n) \leq cn \log_2 n$ for all $n \geq n_0$ for $c = 5$ and $n_0 = 2$
- So we've proved $T(n) = O(n \log n)$, which was our original goal
  - But why did we take $c = 5$ and $n_0 = 2$ for our proof?

Warning!

- Important not to use asymptotic notation in the inductive hypothesis!
- Consider our recurrence again: $T(n) = 2T(n/2) + n$
  - We assumed $T(n) \leq 5n \log_2 n$. We did not assume $T(n) = O(n \log_2 n)$
- Otherwise, it looks like you can prove $T(n) = O(n)$ for the recurrence
  - Inductive hypothesis: $T(n) = O(n)$
  - Induction step: $T(n) = 2(O(n/2)) + \theta(n)$
    $= O(n)$

WRONG!